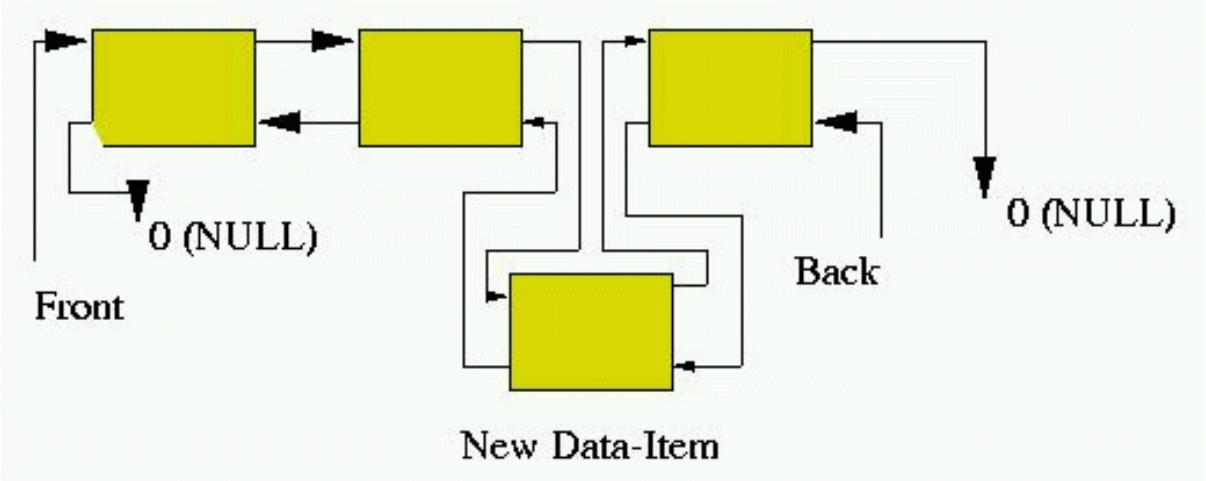
STL List

The STL list class is implemented as a *doubly linked list*

Each node in the list maintains a pointer to the next *and* previous nodes.

The following diagram illustrates the general concept of a doubly linked list:



The list is a sequence, and is bidirectional, but it is not random access.

The only way to access items inside of a linked list is by using iterators or the front() and back() functions defined by the sequence container.

Example:

```
#include <list>

…

using namespace std;

list<int> listObject;

list<int>::iterator iterStart;

// fill the list so that it has 20,10,50,40

listObject.push_back(50);

listObject.push_back(40);

listObject.push_front(10);

listObject.push_front(20);

// sort the list using the list::sort() function
```

listObject.sort();

// multiply each item by 2:

list<int>::iterator iterEnd = listObject.end();

for (iterStart = listObject.begin(); iterStart != iterEnd; ++iterStart) {

(*iterStart) *= 2;

}

Splicing list elements

To splice means to join or unite.

Only std::list offers the ability to splice elements from one place to another without copying any data.

Three different prototypes for splice()

void splice (iterator position, list<T>& x);

void splice (iterator position, list<T>& x, iterator i);

void splice (iterator position, list<T>& x, iterator first, iterator last);

The *first version (1)* transfers all the elements of *x* into the container.
The *second version (2)* transfers only the element pointed by *i* from *x* into the container.
The *third version (3)* transfers the range `[first,last)` from *x* into the container.

splice() moves elements from list *x* into the list container at the specified *position*, effectively inserting the specified elements into the container and removing them from *x*.

This increases the container size by the amount of elements inserted, and reduces the size of *x* by the same amount (whenever *x* is not the same as *this).

The operation does not involve the construction or destruction of any element object

The iterators that pointed to moved elements are no longer valid.

Example

```
// splicing lists
#include <iostream>
#include <list>
```

```
int main ()
{
  std::list<int> mylist1, mylist2;
  std::list<int>::iterator it;

  // set some initial values:
  for (int i=1; i<=4; ++i)
     mylist1.push_back(i);      // mylist1: 1 2 3 4

  for (int i=1; i<=3; ++i)
     mylist2.push_back(i*10);   // mylist2: 10 20 30

  it = mylist1.begin();
  ++it;                         // points to 2

  mylist1.splice (it, mylist2); // mylist1: 1 10 20 30 2 3 4
                                // mylist2 (empty)
                                // "it" still points to 2 (the 5th
element)

  mylist2.splice (mylist2.begin(),mylist1, it);

                                // mylist1: 1 10 20 30 3 4
                                // mylist2: 2
                                // "it" is now invalid.
  it = mylist1.begin();
  it++;
  it++;
  it++;
  // "it" points now to 30

  mylist1.splice ( mylist1.begin(), mylist1, it, mylist1.end());
                                // mylist1: 30 3 4 1 10 20
                                // mylist2: 2

  std::cout << "mylist1 contains:";
  for (it=mylist1.begin(); it!=mylist1.end(); ++it)
    std::cout << ' ' << *it;
  std::cout << '\n';

  std::cout << "mylist2 contains:";
  for (it=mylist2.begin(); it!=mylist2.end(); ++it)
    std::cout << ' ' << *it;
  std::cout << '\n';

  return 0;
}
```

Output:

```
mylist1 contains: 30 3 4 1 10 20
mylist2 contains: 2
```

Checking for an empty list

There are two ways to check for an empty list:

if (listObject.size() == 0) {

or

if (listObject.empty()) {

empty() runs in O(c) time for all standard containers

size() runs in O(n) time for some implementations

list::size() does not offer constant time performance because of the way the list::splice() operation is implemented.

STL Maps

Before we discuss maps, in passing note that STL provides both a queue class (`std::queue`) and a stack class (`std::stack`)

Include the appropriate library:

```
#include <queue>
```

```
#include <stack>
```

Read more about STL stacks/queues here:

http://www.cplusplus.com/reference/stl/queue/

In basic terms a Map is a large cache on which we can retrieve information quickly

Formally, a Map (or table) is an object that maps keys to values

The keys are unique and each key maps to at most one value

Applications of maps in computing?

e.g. DNS server: host names to IP addresses

Each entry in a map is a pair, one of whose fields is designated as the key field

No two entries in a map can have the same key

However, two or more keys can map to the same value

C++ and maps

C++ provides a `std::map` which is implemented using a form of binary tree

The keys are sorted and must be comparable using operator <

C++ 11 provides `std::unordered_map` which uses a hash table implementation

Keys are not sorted

Tutorial

http://www.cprogramming.com/tutorial/stl/stlmap.html

The `std::map` class implements a (sorted) associative array.

Before map containers can be used the following preprocessor directive is required:

```
#include <map>
```

A map is filled with *key/value* pairs, which may be of any container-acceptable type.

Types are associated with both the key and the value, we must specify two types in the angular bracket notation.

Example:

```
map<string, double> object;
```

A map sorts its keys thus the key's operator<() must be defined

E.g. if the key represents an object, we must be able to compare keys using <

Bad idea to use pointers for keys…sorting pointers is different than sorting what they point at!

Two fundamental operations on STL maps are:

storage of *Key/Value* combinations

the retrieval of values (given their keys)

the *Key/Value* is STL is referred to as a pair.

The index operator [] (using a key as the index), can be used for both.

If the index operator is used as *lvalue*, insertion will be performed, e.g.

```
map<string, int> freq;

freq["One"] = 1;
```

If it is used as *rvalue*, the key's associated value is retrieved, e.g.

```
int value = freq["One"];
```

Each key can be stored only once in a map.

If the same key is entered again, the new value replaces the formerly stored value, which is lost.

```
freq["One"] = 134;
```

You can remove a key value pair from the map by:

```
freq.erase("One");
```

You can find

```
map<string, int>::iterator iter = freq.find(“One”);
```

find() returns an interator to the found pair if it is in the map, if not it returns freq.end()

iter->first returns the key

iter->second returns the value

NOTE:

The preferred way to insert into a map is to use the insert() operation

So, instead of:

```
freq[“one”]=3;
```

Could try:

```
freq.insert(make_pair(“one”, 3));
```

Beware that insert() will NOT insert a new pair if the key already exists in the map

To check this, we can pick up the return value from insert – a pair object containing a map iterator and a boolean value:

```
pair<map<string, int>::iterator, bool> result =
        freq.insert(make_pair(“One”, 1));

if(result.second == false) {
        // If insert failed (key already exists)….

        }
```

Example use of STL Map

```
#include <iostream>
#include <map>
#include <string>
#include <cstdlib> // For rand() and srand()
#include <ctime>   // For seeding rand()

// Function to generate a random short key
std::string generateShortKey() {
    const char charset[] =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
```

```
    const int keyLength = 6; // Short key length
    std::string key;

    for (int i = 0; i < keyLength; ++i) {
        key += charset[rand() % (sizeof(charset) - 1)];
    }
    return key;
}
// Function to generate a unique short key
std::string generateUniqueKey(std::map<std::string, std::string>& urlMapping) {
    std::string newKey;
    do {
        newKey = generateShortKey(); // Generate a random key
    } while (urlMapping.count(newKey) > 0); // Check if the key already exists
    return newKey;
}
int main() {
    // Seed the random number generator
    srand(static_cast<unsigned>(time(0)));

    // Map to store short URL -> long URL mappings
    std::map<std::string, std::string> urlMapping;

    // Define some long URLs
    std::string longUrl1 = "https://www.example.com/some/long/url";
    std::string longUrl2 = "https://www.anotherexample.com/very/long/path";
```

```
// Generate short keys for the URLs and store them in the map
std::string shortUrl1 = generateUniqueKey(urlMapping);
std::string shortUrl2 = generateUniqueKey(urlMapping);

urlMapping[shortUrl1] = longUrl1;
urlMapping[shortUrl2] = longUrl2;



std::cout << "URL Mappings:\n";

std::map<std::string, std::string>::iterator it;

for (it = urlMapping.begin(); it != urlMapping.end(); it++)
{
    std::cout << it->first    // string (key)
        << ':'
        << it->second   // string's value
        << std::endl;
}



// Allow the user to input a short URL and retrieve its original long URL
std::cout << "\nEnter a short URL to retrieve its original URL: ";
std::string userInput;
std::cin >> userInput;
```

```
    if (urlMapping.count(userInput)) {
        std::cout << "Original URL: " << urlMapping[userInput] << '\n';
    }
    else {
        std::cout << "Short URL not found.\n";
    }

    return 0;
}
```

Exercise : What does it do?

```cpp
#include <map>
#include <iostream>
#include <string>
using namespace std;

int  main()
{

	map<string, int> freq;


	string numbers[] = { "zero", "one", "two", "three", "four", "five"
};
	int number = 12345033;

	for (int i = 0; i <= 7; i++) {
		int j = number % 10;

		map<string, int>::iterator iter = freq.find(numbers[j]);
		int frequency;
		if (iter == freq.end())
		{
			frequency = 0;
		}
		else{
			frequency = iter->second;
		}
		freq[numbers[j]] = ++frequency;
		number = number / 10;
	}
	map<string, int>::iterator iter = freq.begin();
	map<string, int>::iterator end = freq.end();

	for (; iter != end; ++iter) {
		cout << iter->first << " = " << iter->second << endl;
	}
}
```

Rewritten original for loop so that the map insert() function is now used instead of the index operator.

```
for (int i = 0; i <= 7; i++) {
	int j = number % 10;

	pair<map<string, int>::iterator, bool> result =
		freq.insert(make_pair(numbers[j], 1));

	if (result.second == false) {
		result.first->second++;
	}
	number = number / 10;

}
```